components can interact easily, but you can't use them with Tag Helpers or view components, and it's hard to combine them with Razor Page form posts.

Nevertheless, if you need an island of rich client-side interactivity in a single Razor Page, you can embed a Blazor component in a Razor Page, as shown in the "Render components from a page or view" section of the "Prerender and integrate ASP.NET Core Razor components" documentation at http://mng.bz/PPen. You could also use Blazor components as a way to replace Asynchronous JavaScript and XML (AJAX) calls in your Razor Pages, as I show in my blog entry "Replacing AJAX calls in Razor Pages with Razor Components and Blazor" at http://mng.bz/9MJj.

If you don't need the client-side interactivity of Blazor, view components are still the best option for isolated sections in Razor Pages. They interoperate cleanly with your Razor Pages; have no additional operational overhead; and use familiar concepts like layouts, partial views, and Tag Helpers. For more details on why you should continue to use view components, see my "Don't replace your View Components with Razor Components" blog entry at http://mng.bz/1rKq.

In this section you'll see how to create a custom view component for the recipe app you built in previous chapters, as shown in figure 32.4. If the current user is logged in, the view component displays a panel with a list of links to the user's recently created recipes. For unauthenticated users, the view component displays links to the login and register actions.